

Fantasy Premier League Points Forecasting and Squad Optimization

Project status report

Generated 2026-07-03

Abstract. This report summarizes the current state of a Fantasy Premier League (FPL) points-prediction and squad-optimization system, originally developed as a Master’s thesis (LSTM forecasting in R combined with a hand-written MILP squad optimizer) and since rewritten into a single, weekly-runnable Python pipeline. It documents the pipeline’s architecture, the evaluation methodology used to validate changes, the results of several forecasting-technique experiments conducted against the production system (all reported honestly, including the negative results), and two exploratory branches investigating alternative approaches. The project’s own backtesting shows the current 6-model ensemble improves on the original LSTM system by approximately 25% in realized squad points over an identical 31-game-week backtest window.

1 Introduction

This project predicts Fantasy Premier League (FPL) player points and uses those predictions to recommend weekly squad decisions (transfers, starting lineup, captaincy, and chip usage). It began as a Master’s thesis combining an LSTM forecasting model (implemented in R/Keras) with a mixed-integer linear program (MILP) for squad selection, based on the formulation of Kristiansen et al. (2018). The system has since been rewritten into a single Python pipeline (the `fpl` package) so that it can be run weekly during a live season rather than only as a one-off academic validation.

This report documents the system’s current architecture, the empirical work done to validate and harden it, and the outcome of several forecasting-technique experiments run against the production pipeline. Where an experiment failed to improve on the existing system, that is reported explicitly - the guiding principle for this phase of the project was to test candidate improvements honestly rather than assume a “more sophisticated” or “more principled” technique must help.

2 Data

Historical data is sourced from Vaastav Anand’s public FPL data repository (`vaastav/Fantasy-Premier-League` on GitHub), which mirrors the official FPL API’s per-gameweek player statistics back to the 2016-17 season. This project uses data from the 2022-23 season onward (four seasons at the time of writing: 2022-23 through 2025-26).

Gameweeks are indexed with a single ascending counter (`GW_global`) across all seasons rather than resetting to 1 each season, since a raw `GW` value is ambiguous without knowing which season it belongs to. Season N occupies gameweeks $(N - 1) \times 38 + 1$ through $N \times 38$.

An exploratory data analysis (`notebooks/eda.ipynb`) was conducted covering the distribution of `total_points` by position and season. The key finding, consistent with domain expectations, is that `total_points` is heavily right-skewed and zero-inflated: most player-gameweek observations score between 0 and 2 points, with an occasional large-value outlier from a goal, hat-trick, or high bonus-point haul. A Shapiro-Wilk test rejects normality decisively at every position (starters only, $p < 0.001$ throughout), and an Augmented Dickey-Fuller test on the average points per gameweek per position indicates the series' mean level is reasonably stable across a season. This distributional shape - not merely a stylistic preference - is the reason the mean absolute scaled error (MASE), rather than mean absolute error (MAE) alone, is used to judge forecast quality throughout this project (see Section 3.4).

3 Forecasting methodology

3.1 Feature engineering

Features are shifted rolling-window and season-to-date statistics per player (3- and 5-gameweek rolling means, and an expanding season-to-date mean, over roughly twenty per-gameweek statistics including points, minutes, expected goals/assists, bonus points system score, and market value). Every feature is shifted by one gameweek relative to its target row, so that no row's features can see that row's own outcome - a standard no-leakage discipline for supervised time-series forecasting.

3.2 Per-position ensemble

Rather than a single global model, four independent models are trained - one per playing position (goalkeeper, defender, midfielder, forward) - since the statistics that matter for predicting a goalkeeper's points (saves, clean sheets) are largely disjoint from those that matter for a forward's (shots, expected goals), and pooling all positions into one model risks one position's scale dominating the loss function.

At each position, six computationally cheap model types are trained and compared: LightGBM (gradient-boosted trees), Ridge regression, ElasticNet, Random Forest, Extra Trees, and k -nearest neighbors. Tree-based models receive raw features including missing values (a player's first few gameweeks have no rolling history yet), since LightGBM and the scikit-learn tree ensembles handle missing values without imputation. Linear and distance-based models are wrapped in a median imputer and standard scaler, since those estimators require complete, scaled input.

The six models' predictions are blended into a single ensemble per position using non-negative least squares (NNLS) regression, which finds non-negative weights (summing to one) minimizing squared error against the true outcome. Critically, these blend weights are fit on one half of a held-out validation window and the ensemble's reported accuracy is evaluated on the **other** half - a genuine holdout, ensuring the reported ensemble accuracy is not simply the result of fitting weights and evaluating them on the same rows used to choose them.

3.3 Evaluation methodology

Two complementary evaluation approaches are used:

1. **Static split:** models are trained on gameweeks up to a cutoff and evaluated on a fixed, contiguous held-out window immediately following it - specifically the same GW77-107 window (2024-25 season, gameweeks 1-31) that the original LSTM system was validated on, so the two approaches are directly comparable.
2. **Walk-forward (expanding-window) validation:** for each gameweek from a starting point onward, a model is trained on strictly earlier data and evaluated only on that single gameweek, then the training window expands by one gameweek and the process repeats. This gives a far larger number of genuinely out-of-sample evaluation points than a single fixed split, at the cost of retraining many times.

3.4 Error metrics

Mean absolute error (MAE) alone is difficult to interpret for an intermittent series like FPL points: an MAE of 1.2 looks identical whether the underlying target typically scores 8 points or typically scores 0. The mean absolute scaled error (MASE; Hyndman & Koehler, 2006) addresses this by expressing forecast error relative to a naive one-gameweek-lag benchmark, with the benchmark’s own error computed strictly on the training window (so the scale itself cannot leak information from the evaluation window it is used to judge). A MASE below 1 indicates the model outperforms the naive “same as last gameweek” forecast; a MASE above 1 indicates it does not, independent of the target’s raw point-scale.

4 Squad optimization

Given a predictions CSV, `fpl/milp/optimize.py` recommends a squad using a mixed-integer linear program closely following the formulation of Kristiansen et al. (2018), which is also the lineage underlying Venter & van Vuuren (2024) - a paper reviewed in full during this project (see References). The model is re-solved every gameweek over a configurable lookahead horizon in a rolling-horizon fashion, with only the first gameweek’s decision locked in before rolling forward. Constraints include:

- a 15-player squad of exactly 2 goalkeepers, 5 defenders, 5 midfielders, and 3 forwards;
- a total squad value not exceeding a fixed budget;
- no more than 3 players from any single real-world club;
- an 11-player starting lineup satisfying formation constraints (exactly 1 goalkeeper, at least 3 defenders, at least 1 forward);
- a captain (2x points) and vice-captain, with automatic substitution when a starter is unused;
- a limited number of free transfers per gameweek, with a 4-point penalty per additional transfer;
- wildcard, free-hit, bench-boost, and triple-captain chip logic, each usable at most once (twice for wildcard) per season and mutually exclusive within a gameweek.

The objective maximizes the sum of predicted lineup points plus captain bonus, less any transfer penalties incurred, summed across the lookahead horizon.

5 Results

5.1 Backtesting reference point

The pipeline was validated against the original LSTM+R system by running both through the same MILP optimizer on the same GW77-107 backtest window (2024-25 season, gameweeks 1-31), using actual realized points rather than forecasts to score the resulting squads:

System	Actual points, GW77-107
Old LSTM + MILP	1526
New LightGBM + MILP	1811
New 6-model ensemble + MILP	1900

The 6-model ensemble improves on the original LSTM system by approximately 25% in realized squad points over an identical evaluation window and identical optimizer.

5.2 Forecasting-technique experiments

Following a full reading of Venter & van Vuuren (2024) - whose §4 MILP formulation was found to match the existing optimizer almost exactly, confirming no further optimizer work was warranted - project focus shifted entirely to forecasting quality. Venter & van Vuuren’s own case study credits full-season lookahead and forecast quality, not MILP sophistication, for their result (top 4.08% of roughly 8.24 million FPL managers in the 2020-21 season using a broadly similar optimizer). Several additional forecasting techniques flagged by that paper as relatively strong individual performers were implemented and tested against the production ensemble, honestly and without assuming a “more principled” technique must help:

Position	Ensemble	OLS (index)	Baseline ¹	SES	Theta	Croston	AR(1)	ARIMA
GK	0.64	0.66	0.69	0.68	0.70	0.89	0.87	0.86
DEF	0.87	0.90	0.94	0.90	0.91	1.06	1.06	1.10
MID	0.87	0.93	0.98	0.94	0.95	1.10	1.11	1.14
FWD	1.07	1.16	1.13	1.11	1.14	1.39	1.29	1.36

MASE, GW77-107 static split. Lower is better; bold marks the ensemble, the best performer at every position.

Naive drift and Holt’s linear trend were also tested (see `RESEARCH_LOG.md` for full figures) and likewise underperformed the existing baseline everywhere. Across all seven simple techniques tested: **simple exponential smoothing (SES) is the best of the simple baselines, but nothing tested beats the full ensemble at any position.** Croston’s method - specifically designed for intermittent-demand series and flagged in the source literature as a strong performer - was the worst performer here, because pooling it across an entire position (rather than selecting it only for the specific players whose scoring pattern it suits, as the source literature actually did) hides the per-player heterogeneity that made it work in the original study. This motivated a subsequent planning exercise into per-player model selection (Section 6.2), rather than per-position pooling, as the more promising remaining direction.

Plain OLS regression as the designated index. On the recommendation that a simple, unregularized multiple linear regression should serve as the project’s designated benchmark - “index” - that every other technique must clear, exactly as a passive market index is the bar an active strategy has to beat, plain OLS regression on the full approximately 70-feature set was added to the comparison (`fpl/model/models.py`). It turns out to be a genuinely strong baseline: better than every time-series technique in the table above at every position, trailing only the regularized linear models (Ridge, ElasticNet) by a small margin - expected, since the engineered feature set includes many mutually correlated rolling-window statistics that unregularized OLS cannot down-weight the way Ridge/ElasticNet can. The production ensemble beats this index

¹3-gameweek rolling mean, falling back to season-to-date mean.

at every position (by 0.02-0.09 MASE), which is a reassuring result: it confirms the ensemble’s added complexity is earning its keep against a genuinely competitive simple benchmark, not merely against a weak straw man.

5.3 Optimizer constraint testing

A regression test (`tests/test_optimize_constraints.py`) verifies that the MILP optimizer’s output always satisfies its own constraints - squad composition, budget, club limits, and starting-lineup formation - using a small synthetic predictions dataset. This is the project’s first automated test; it was confirmed to actually catch regressions by temporarily loosening a constraint in the optimizer and observing the test fail before reverting the change.

6 Parallel exploratory branches

Two lines of investigation were pursued on isolated git branches, deliberately separated from the production pipeline so that speculative or higher-risk work could not entangle or block it.

6.1 Bayesian belief-state MDP manager

Matthews, Ramchurn & Chalkiadakis (2012) frame FPL as a belief-state Markov decision process, maintaining Bayesian beliefs over each player’s starting probability, goal probability, and assist probability, updated via conjugate-prior rules as results are observed, with team selection posed as a multi-dimensional knapsack problem and solved via either a myopic (single-gameweek) policy or a Q-learning policy with a bounded candidate-action pool. An implementation of this approach (`bayesian_manager/`, on branch `experimental/bayesian-mdp-manager`) was backtested on the identical GW77-107 window:

Manager	Actual points, GW77-107
Myopic (single-gameweek) policy	1083
Q-learning candidate-pool policy	847
Production ensemble + MILP (reference)	1900

Both variants score well below the production system, which is expected given several necessary simplifications relative to the source paper: a proxy for player-absence detection (using zero minutes rather than a curated injury/suspension list), a simplified discrete-time club-level goal model in place of the paper’s continuous-time birth process, no simulation of bonus points, cards, or saves, and a one-step-lookahead approximation in place of full multi-step search. This branch is not merged and does not affect the production pipeline.

6.2 Per-player forecasting model selection

Venter & van Vuuren (2024) select, for each individual player, whichever of roughly fifteen candidate forecasting methods (including several tested in Section 3.4’s table) performs best for that specific player by cross-validated MASE - rather than pooling one method across an entire position, as this project’s own experiments above did. A planning exercise (not an implementation) was conducted to assess the feasibility of this approach for this project’s data (`PER_PLAYER_MODEL_SELECTION_PLAN.md`, branch `worktree-agent-afeff933546ce7d37`). Its key finding: approximately 35% of players active in the current season have no prior-season history at all, and the median prior history among players who do have some is only 38 gameweeks - capping the technique’s plausible reach at roughly 60-65% of the live player pool even before results are considered, since per-player cross-validated model selection is unreliable on very short series. The plan’s recommendation is a cautious, pilot-scale go (a single-position

pilot before any larger commitment), noting it is plausible - and would itself be a legitimate, cheap-to-detect negative result - that per-player selection mostly just re-selects the existing ensemble anyway, consistent with the pattern established across Section 3.4’s experiments.

7 Discussion and limitations

The forecasting experiments conducted so far establish a consistent pattern: the existing 6-model, per-position ensemble outperforms every simpler alternative tested at every position. This is a meaningful finding in itself - it means further effort spent adding individual “more sophisticated” model types to the existing architecture is unlikely to yield further gains, and that any future improvement is more likely to come from a genuine architectural change (such as per-player rather than per-position model selection) than from adding another candidate model type to the existing pool.

The forward (FWD) position remains the hardest to forecast across every technique tested (MASE consistently above 1.0, i.e. worse than a naive last-gameweek forecast, for several of the simpler baselines), consistent with forwards having the highest-variance, most “boom-or-bust” scoring profile of the four positions.

A known limitation of the live weekly driver (`fpl/run_week.py`) is that its fixture and current-squad fetching from the official FPL API can only be exercised end-to-end once a season is set up on the FPL website (typically a few weeks before gameweek 1); this had not been tested live as of this writing, since the 2026-27 season had not yet opened.

8 Conclusion and future work

This project has been rewritten from a one-off Master’s thesis validation into a maintainable, weekly-runnable Python pipeline that measurably outperforms its LSTM-based predecessor (an approximately 25% improvement in realized backtest points). This phase of work added a proper error metric (MASE), a regression test suite, and tested seven alternative forecasting techniques against the production system - honestly reporting that none beat it, which is itself useful information ruling out a class of “add more model types” improvements. Two exploratory branches (a Bayesian belief-state MDP manager and a per-player model-selection plan) remain available for future investigation without having touched the production pipeline.

The most promising concrete next step identified is a small-scale pilot of per-player forecasting model selection at a single position, to test - cheaply - whether the heterogeneity that this project’s position-pooled experiments could not exploit is in fact present and exploitable at the individual-player level.

9 References

- Croston, J. D. (1972). Forecasting and stock control for intermittent demands. *Journal of the Operational Research Society*, 23(3), 289-303.
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679-688.
- Kristiansen, B. K., Gupta, A., & Eilertsen, W. (2018). *Developing a forecast-based optimisation model for Fantasy Premier League* (MSc thesis). Norwegian University of Science and Technology, Trondheim.
- Matthews, T., Ramchurn, S. D., & Chalkiadakis, G. (2012). Competing with humans at fantasy football: Team formation in large partially-observable domains. *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 1394-1400.
- Venter, V., & van Vuuren, J. H. (2024). An optimisation approach towards soccer Fantasy Premiere League team selection. *ORiON*, 40(1), 69-107.